DAY 4 · ERROR HANDLING & MODULES

# ASSIGNMENT — 20 QUESTIONS

Geeta University · Summer Internship 2026

| 20 | 30 min | MCQ | 4 — Errors & Modules |
|---|---|---|---|
| Total Marks | Time | Type | Day |

**Instructions**

Har question ke liye ek sahi answer choose karo. Sab questions 1 mark ke hain.

★ questions interview mein frequently pooche jaate hain — carefully solve karo.

Code-trace questions mein mentally execute karo — console mat kholna!

**Q1.** [try/catch/finally] ★ finally block kab execute hota hai?

**(A)** Sirf jab try block successfully complete ho

**(B)** Sirf jab error aaye

**(C)** Hamesha — chahe try succeed kare ya catch handle kare

**(D)** Sirf jab return statement ho

**Q2.** [try/catch/finally] ★ finally mein return statement ho to kya hoga?

**(A)** try ka return use hoga

**(B)** finally ka return try/catch dono ke return ko override kar deta hai

**(C)** Error throw hoga

**(D)** undefined return hoga

**Q3.** [try/catch/finally] catch(e) mein 'e' mein kya hota hai?

**(A)** Sirf error message string

**(B)** Error object — name, message, stack properties ke saath

**(C)** Error type number

**(D)** Boolean false

**Q4.** [try/catch/finally] ★ Empty catch block catch(e){} kyun avoid karna chahiye?

**(A)** Performance impact hota hai

**(B)** Error silently swallow ho jaata hai — debugging impossible

**(C)** Syntax error hai

**(D)** finally run nahi hota

**Q5.** [try/catch/finally] Error re-throw kab karna chahiye?

**(A)** Jab hum handle karna chahte ho

**(B)** Jab current context mein error properly handle nahi ho sakta — caller ko handle karne do

**(C)** Hamesha re-throw karo

**(D)** Kabhi nahi

**Q6.** [Custom Errors] Custom Error class mein super(message) call karna kyon zaroori hai?

**(A)** Performance ke liye

**(B)** Error.prototype ka message property set karne ke liye

**(C)** Stack trace banana ke liye

**(D)** Optional hai — zarori nahi

**Q7.** [Custom Errors] ★ instanceof se specific error types catch karne ka kya faida hai?

**(A)** Koi faida nahi — generic catch theek hai

**(B)** Alag error types ko alag tarah handle kar sakte hain — targeted response

**(C)** Faster execution hoti hai

**(D)** Multiple catch blocks likh sakte hain

**Q8.** [Custom Errors] this.name = this.constructor.name; — ye kya karta hai custom error mein?

**(A)** Error ko class ka naam assign karta hai automatically

**(B)** Error ka message set karta hai

**(C)** Stack trace clear karta hai

**(D)** Nothing — optional line hai

**Q9.** [ES Modules] ★ Named export import karne ka sahi syntax kaunsa hai?

**(A)** import add from './math.js'

**(B)** import { add } from './math.js'

**(C)** import * add from './math.js'

**(D)** require('./math.js').add

**Q10.** [ES Modules] Default export import karne ka sahi syntax?

**(A)** import { Logger } from './logger.js'

**(B)** import * as Logger from './logger.js'

**(C)** import Logger from './logger.js'

**(D)** import default Logger from './logger.js'

**Q11.** [ES Modules] ★ Ek file mein kitne default exports ho sakte hain?

**(A)** Unlimited

**(B)** 2

**(C)** Sirf 1

**(D)** Depends on file size

**Q12.** [ES Modules] Namespace import ka sahi syntax kaunsa hai?

**(A)** import { * } from './math.js'

**(B)** import * as MathUtils from './math.js'

**(C)** import all from './math.js'

**(D)** import namespace from './math.js'

**Q13.** [ES Modules] ★ Barrel file (index.js) kyun use karte hain?

**(A)** Performance improve hoti hai

**(B)** Single entry point — consumers ko internal structure jaanna nahi padta

**(C)** Code size reduce hota hai

**(D)** Tree shaking disable karne ke liye

**Q14.** [CJS vs ESM] ★ CommonJS aur ESM mein sabse bada farq kya hai?

**(A)** CJS sirf browser mein kaam karta hai

**(B)** CJS synchronous loading, ESM static/async — tree shaking ESM mein possible

**(C)** ESM purana hai

**(D)** CJS zyada features deta hai

**Q15.** [CJS vs ESM] Node.js mein .mjs extension ka kya matlab hai?

**(A)** Minified JavaScript

**(B)** Always treated as ES Module — regardless of package.json

**(C)** Module JavaScript — koi fark nahi

**(D)** Mobile JavaScript

**Q16.** [CJS vs ESM] ESM mein __dirname available nahi hota — iski jagah kya use karte hain?

**(A)** process.cwd()

**(B)** fileURLToPath(import.meta.url) se derive karte hain

**(C)** path.resolve('./')

**(D)** __dirname ESM mein bhi available hai

**Q17.** [Dynamic import] ★ Dynamic import() kya return karta hai?

**(A)** Module object directly

**(B)** Promise jo module object ke saath resolve hoti hai

**(C)** undefined

**(D)** String path

**Q18.** [Dynamic import] Dynamic import ka main benefit kya hai?

**(A)** Synchronous loading

**(B)** Code splitting — on-demand loading, initial bundle size chota

**(C)** Better error handling

**(D)** TypeScript support

**Q19.** [Bundling] Tree shaking kya hai?

**(A)** Dead code removal — unused exports automatically bundle se nikal jaate hain

**(B)** File compression

**(C)** Code minification

**(D)** Module splitting

**Q20.** [Bundling] ★ Vite development mode mein bundling kyun nahi karta?

**(A)** Vite incomplete tool hai

**(B)** Native browser ESM use karta hai — each file as separate module, no bundling needed in dev

**(C)** Bundling slow hota hai

**(D)** Only Webpack bundle karta hai

## Answer Key

| | | | | |
|---|---|---|---|---|
| **Q1:** (C) | **Q2:** (B) | **Q3:** (B) | **Q4:** (B) | **Q5:** (B) |
| **Q6:** (B) | **Q7:** (B) | **Q8:** (A) | **Q9:** (B) | **Q10:** (C) |
| **Q11:** (C) | **Q12:** (B) | **Q13:** (B) | **Q14:** (B) | **Q15:** (B) |
| **Q16:** (B) | **Q17:** (B) | **Q18:** (B) | **Q19:** (A) | **Q20:** (B) |

### Scoring Guide

20/20 (100%) — Excellent! Day 5 DOM ke liye fully ready.

15-19 (75-95%) — Good! Error handling aur module syntax ek baar aur practice karo.

10-14 (50-70%) — Theory notes dubara padhein, especially CJS vs ESM section.

Below 10 (<50%)— Mentor se mil lo — ye concepts Week 2+ mein bhi heavily use honge.